Chapter 15: Concurrency Control

Database Management Systems — Korth, 6th Edition

---

1. Introduction to Concurrency

Concurrency in a database management system refers to allowing many transactions to access the same database simultaneously. While this is essential for performance and user throughput, it introduces the risk of interference between transactions that may corrupt data consistency.

To handle this, a control mechanism is needed to ensure that concurrent transactions do not interfere with each other. The primary goal is to develop concurrency control protocols that assure serializability — guaranteeing that the outcome of concurrent execution is equivalent to some serial execution of the same transactions.

Key objectives of concurrency control:

- Maximize concurrent access to data
- Prevent transactions from interfering with each other
- Ensure the database remains in a consistent state
- Guarantee conflict-serializable schedules

---

2. Three Concurrency Problems

The logical possibility of data corruption demands the need for concurrency control. These problems arise whenever two or more transactions simultaneously read or write on the same part of the same database. There are three classic problems:

- Lost Update
- Uncommitted Dependency (Dirty Read)
- Inconsistent Analysis

2.1 Lost Update Problem

The Lost Update problem occurs when two transactions read the same data item and then both update it. The second write overwrites the first without ever reading the first transaction's updated value, causing the first update to be silently lost.

Example Scenario (t = a tuple in a table):

Time	User 1 (Trans A)	User 2 (Trans B)
1	Retrieve t	
2		Retrieve t
3	Update t	
4		Update t ← OVERWRITES Trans A!

Explanation: Transaction A reads tuple t at time 1 and modifies it at time 3. Meanwhile, Transaction B also reads t at time 2 (before A's write is visible), and overwrites t at time 4 without considering A's change. Transaction A's update is completely lost.

2.2 Uncommitted Dependency (Dirty Read)

The Uncommitted Dependency problem occurs when one transaction reads a value that has been written by another transaction that has not yet committed. If that other transaction rolls back, the first transaction has operated on data that never truly existed.

Example Scenario:

Time	User 1 (Trans A)	User 2 (Trans B)
1		Update t
2	Retrieve t (reads dirty data)	
3		Rollback! (Trans A has bad data)

Explanation: Transaction A reads tuple t (time 2) after Transaction B has updated it (time 1) but before B commits. When B rolls back (time 3), A holds a value that is now invalid. Any computation A makes based on this value will be incorrect.

2.3 Inconsistent Analysis Problem

The Inconsistent Analysis problem (also called a Non-Repeatable Read or Phantom) occurs when a transaction reads multiple data items while another transaction is concurrently modifying them. The reading transaction sees a mix of old and new values, producing an internally inconsistent view.

Initial State: Acc1 = 40, Acc2 = 50, Acc3 = 30 (Total = 120)

Time	User 1 (Trans A) — Summing	User 2 (Trans B) — Transferring

1	Retrieve Acc1: Sum = 40	
2	Retrieve Acc2: Sum = 90	
3	Retrieve Acc3: ...	
4		Update Acc3: 30 → 20
5		Retrieve Acc1
6		Update Acc1: 40 → 50
7		Commit
8	Retrieve Acc3: Sum = 110 (WRONG! Expected 120)	

Explanation: Transaction A is computing a sum of accounts. Meanwhile, Transaction B transfers 10 from Acc3 to Acc1. Because A reads Acc3 before the transfer (gets 30) and reads Acc1 after the transfer (gets 50), it arrives at a sum of 40 + 50 + 20 = 110 instead of the correct 120. The analysis is inconsistent.

---

3. Lock-Based Protocols

Locks are the primary mechanism used to control concurrent access to data items. By requiring transactions to acquire locks before accessing data, the DBMS can serialize conflicting operations and prevent the concurrency problems described above.

3.1 Types of Locks

Data items can be locked in two fundamental modes:

1. Exclusive (X) Mode: The data item can be both read and written. An X-lock is requested using the lock-X instruction. Only one transaction may hold an X-lock on a data item at a time.
2. Shared (S) Mode: The data item can only be read. An S-lock is requested using the lock-S instruction. Multiple transactions may simultaneously hold S-locks on the same data item.

Lock requests are made to the concurrency-control manager. A transaction can proceed only after its lock request is granted.

3.2 Lock Compatibility Matrix

The compatibility matrix determines whether a requested lock can be granted given the locks already held by other transactions:

	S (Req.)	X (Req.)
S (Grant)	true	false
X (Grant)	false	false

Rules:

- Any number of transactions can hold shared (S) locks on the same data item simultaneously (S-S compatible = true).
- If any transaction holds an exclusive (X) lock, no other transaction may hold any lock (X is incompatible with everything else).
- If a lock cannot be granted, the requesting transaction is made to wait until all incompatible locks held by other transactions have been released.

3.3 Example Transaction with Locks

The following example shows T1 and T2 performing locking correctly on items A and B:

```
T1:                              T2:
lock-X(B);                       lock-S(A);
read(B);                         read(A);
B := B - 50;                     unlock(A);
write(B);                        lock-S(B);
unlock(B);                       read(B);
lock-X(A);                       unlock(B);
read(A);                         display(A + B)
A := A + 50;
write(A);
unlock(A)
```

A locking protocol is a set of rules followed by all transactions while requesting and releasing locks. These protocols restrict the set of possible schedules to only safe, consistent ones.

---

4. Pitfalls of Simple Lock-Based Protocols

Even with locks, naively releasing them too early creates inconsistencies. Consider the following scenario with A = 1000, B = 500 (sum should be preserved at 1500):

```
T1:                              T2:
```

```
Lock-X(A)                          Lock-S(A)
Read(A)                            Read(A)
A = A - 100                        Unlock-S(A)
Write(A)                           Lock-S(B)
Unlock-X(A)   <-- unlocks here     Read(B)
                                   Unlock-S(B)
                                   Display A + B  <-- T2 reads A=900, B=500 = 1400
(WRONG!)
Lock-X(B)
Read(B)
B = B + 100
Write(B)
Unlock-X(B)
```

Problem: T2 reads A after T1 decrements it (900) but reads B before T1 increments it (500), getting a total of 1400 instead of the correct 1500. The schedule is not serializable.

Key Insight: Locks should be kept until all operations are complete. Unlocking early creates the window for inconsistency. This motivates the Two-Phase Locking Protocol.

---

5. The Two-Phase Locking Protocol (2PL)

The Two-Phase Locking Protocol (2PL) is the most widely used protocol for ensuring conflict-serializable schedules. It divides a transaction's lifecycle into exactly two phases:

5.1 The Two Phases

3. Phase 1 — Growing Phase: The transaction may acquire (obtain) locks but may NOT release any locks.
4. Phase 2 — Shrinking Phase: The transaction may release locks but may NOT acquire any new locks.

The point at which a transaction acquires its final lock is called the lock point. 2PL guarantees that transactions can be serialized in the order of their lock points.

5.2 Example of 2PL

```
T3:                                T4:
lock-X(B);    <- Growing           lock-S(A);    <- Growing
read(B);                           read(A);
B := B - 50;                       lock-S(B);    <- Lock Point for T4
write(B);                          read(B);
lock-X(A);    <- Lock Point T3     display(A + B);
read(A);                           unlock(A);    <- Shrinking
```

```
A := A + 50;                        unlock(B);
write(A);
unlock(B);     <- Shrinking
unlock(A);
```

The serial order guaranteed by lock points in this example: T4 comes before T3 (T4's lock point precedes T3's).

5.3 Lock Conversions

2PL also allows lock conversions within each phase:

- First Phase (Growing): Can acquire S-lock, can acquire X-lock, can upgrade S → X.
- Second Phase (Shrinking): Can release S-lock, can release X-lock, can downgrade X → S.

Downgrading is useful when a transaction no longer needs exclusive write access but still needs to read.

5.4 Formal Definition of Legal Schedules

Let {T0, T1, ..., Tn} be a set of transactions in schedule S. We say Ti precedes Tj (written Ti → Tj) if:

- Ti held lock mode A on item Q, and
- Tj held lock mode B on Q later, and
- comp(A, B) = false (the lock modes are incompatible).

A schedule S is legal under a locking protocol if it is a possible schedule for transactions following that protocol's rules. A locking protocol ensures conflict serializability if and only if all legal schedules are conflict serializable — meaning the associated precedence relation is acyclic.

5.5 Problems with Basic 2PL

While 2PL ensures serializability, it still has three significant drawbacks:

5. Unnecessary Wait Due to Early Locking: A transaction must hold locks from the beginning of its growing phase, even before it needs certain resources, causing other transactions to wait longer than necessary.
6. Deadlock: Two or more transactions can mutually wait for each other's locks, forming a circular dependency from which no transaction can proceed without external intervention.

7. Cascading Rollback: If an uncommitted transaction T fails, any other transaction that has read T's written values must also be rolled back, which may cascade further.

5.6 Variants of 2PL

To address cascading rollbacks, two stricter variants of 2PL were developed:

Feature	Basic 2PL	Strict 2PL	Rigorous 2PL
Exclusive lock release	After shrinking begins	Only at commit/abort	Only at commit/abort
Shared lock release	After shrinking begins	After shrinking begins	Only at commit/abort
Cascading rollback	Possible	Prevented	Prevented
Deadlock-free	No	No	No
Serializability order	Lock points	Lock points	Commit order

Note: None of the 2PL variants prevent deadlocks. Deadlock handling requires separate mechanisms (see Section 7).

---

6. Transaction Isolation Levels

Isolation levels define the degree of interference a transaction is prepared to tolerate from concurrent transactions. If full serializability is guaranteed, interference is zero. However, strict isolation is expensive — lower isolation levels trade some correctness for improved concurrency and throughput.

6.1 The Four Isolation Levels

SQL standard defines four isolation levels in decreasing order of strictness:

SERIALIZABLE > REPEATABLE READ > READ COMMITTED > READ UNCOMMITTED

Level	Description	Interference
Serializable	Completely isolated execution, as if serial	None
Repeatable Read	Re-reading same row yields same value; phantoms possible	Only Phantoms

Read Committed	Only committed data is visible; re-reading may differ	Phantoms + NR
Read Uncommitted	Even dirty (uncommitted) data is visible	All Three

6.2 Issues with Lower Isolation Levels

Lower isolation levels violate serializability in three specific ways:

8. Dirty Read: T1 updates a row; T2 reads the row; T1 rolls back. T2 now holds a value that never existed in committed state.
9. Non-Repeatable Read: T1 reads a row; T2 updates the row and commits; T1 reads the same row again. T1 sees two different values for what it considers the 'same' row.
10. Phantom Read: T1 retrieves all rows matching a condition; T2 inserts a new row satisfying the same condition; T1 repeats its query and sees a row that didn't exist before — a 'phantom.'

6.3 Isolation Level vs. Problem Matrix

Isolation Level	Dirty Reads	Non-repeatable Reads	Phantoms
Read Uncommitted	X ✗	X ✗	X ✗
Read Committed	— ✓	X ✗	X ✗
Repeatable Read	— ✓	— ✓	X ✗
Serializable	— ✓	— ✓	— ✓

X = the isolation level suffers from this problem. — = the isolation level is immune to this problem.

---

7. Deadlock Handling

A system is in a deadlock state if there exists a set of transactions such that every transaction in the set is waiting for another transaction in the same set to release a data item. No transaction can proceed without external intervention.

Example of a simple deadlock:

```
T3: holds lock on B, requests lock on A
T4: holds lock on A, requests lock on B
```

```
=> T3 waits for T4; T4 waits for T3 => Deadlock!
```

There are two main strategies to handle deadlocks: Prevention and Detection.

7.1 Deadlock Prevention Strategies

Deadlock prevention ensures the system never enters a deadlock state. These strategies use transaction timestamps (assigned at start time) to determine priority.

Scheme	Wait-Die (Non-Preemptive)	Wound-Wait (Preemptive)
Type	Non-preemptive	Preemptive
Older Tx	Waits for younger	Wounds (rolls back) younger
Younger Tx	Dies (rolls back)	Waits for older
Starvation	Avoided (original timestamp kept)	Avoided (original timestamp kept)

Wait-Die Scheme (Non-Preemptive)

When Ti requests a data item held by Tj:

- If Ti is older (smaller timestamp) than Tj: Ti waits.
- If Ti is younger (larger timestamp) than Tj: Ti dies (rolls back).

Example with T14 (ts=5), T15 (ts=10), T16 (ts=15):

- T14 requests item held by T15: T14 waits (T14 is older).
- T16 requests item held by T15: T16 dies/rolls back (T16 is younger).

Wound-Wait Scheme (Preemptive)

When Ti requests a data item held by Tj:

- If Ti is older (smaller timestamp) than Tj: Ti wounds Tj (Tj is rolled back preemptively).
- If Ti is younger (larger timestamp) than Tj: Ti waits.

Example with T14 (ts=5), T15 (ts=10), T16 (ts=15):

- T14 requests item held by T15: T15 is rolled back (T14 wounds T15).
- T16 requests item held by T15: T16 waits (T16 is younger).

Common Properties of Both Schemes

- Rolled-back transactions restart with their original timestamp to avoid starvation (older transactions keep priority over newer ones).
- Starvation is avoided because a rolled-back transaction retains its original (older) timestamp on restart.

Timeout-Based Scheme

A simpler alternative: a transaction waits for a lock only for a specified amount of time. If not granted within that time, it rolls back and restarts.

- Pro: Simple to implement; no deadlocks possible.
- Con: Too long a wait = unnecessary delay; too short = unnecessary rollbacks (wasted work). Starvation is possible. The right timeout value is hard to determine.

---

8. Deadlock Detection

Rather than preventing deadlocks, some systems allow them to occur and then detect and break them. Detection uses a data structure called the Wait-For Graph.

8.1 Wait-For Graph

The wait-for graph G = (V, E) is defined as:

- V = Set of vertices, one per active transaction in the system.
- E = Set of directed edges. An edge Ti → Tj means Ti is waiting for Tj to release a data item.

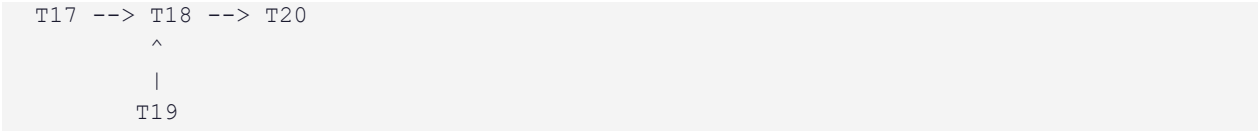
Graph maintenance rules:

- An edge Ti → Tj is inserted when Ti requests an item currently held by Tj.
- The edge is removed when Tj releases the data item needed by Ti.

Deadlock Detection Rule: The system is in a deadlock state if and only if the wait-for graph contains a cycle. A deadlock-detection algorithm must be invoked periodically to check for cycles.

8.2 Wait-For Graph Examples

Without a cycle — No deadlock (T17, T18, T19, T20 form a tree):



```
No cycle => No deadlock
```

With a cycle — Deadlock detected (T18 and T19 wait for each other):

```
  T17 --> T18 --> T20
          ^  \
          |   v
         T19 <-- T18   <== cycle between T18 and T19!

Cycle exists => DEADLOCK
```

8.3 Precedence Graph vs Wait-For Graph

These two graphs serve different purposes in concurrency control:

Aspect	Precedence Graph	Wait-For Graph
Purpose	Test conflict-serializability of a schedule	Detect deadlocks among active transactions
Arc direction	Ti → Tj if Ti accesses X before Tj with conflict	Ti → Tj if Ti is waiting for Tj to release a lock
Cycle meaning	Schedule is NOT conflict-serializable	DEADLOCK exists
Used when	Evaluating a completed or proposed schedule	Monitoring active lock requests at runtime

Precedence Graph Arc Rules (Ti → Tj if):

- Ti reads X before Tj writes X
- Ti writes X before Tj reads X
- Ti writes X before Tj writes X

Wait-For Graph Arc Rules (Ti → Tj if):

- T1 read-locks X, then T2 tries to write-lock it
- T1 write-locks X, then T2 tries to read-lock it
- T1 write-locks X, then T2 tries to write-lock it

8.4 Detailed Example: Building Both Graphs

Given the following operation schedule:

```
Step 1:  T1 Read(X)  — T1 read-locks(X)
```

```
Step 2:  T2 Read(Y)  — T2 read-locks(Y)
Step 3:  T1 Write(X) — T1 write-locks(X)  [upgrade from read-lock]
Step 4:  T2 Read(X)  — T2 TRIES read-lock(X)  => T2 must WAIT for T1
Step 5:  T3 Read(Z)  — T3 read-locks(Z)
Step 6:  T3 Write(Z) — T3 write-locks(Z)  [upgrade]
Step 7:  T1 Read(Y)  — T1 read-locks(Y)
Step 8:  T3 Read(X)  — T3 TRIES read-lock(X)  => T3 must WAIT for T1
Step 9:  T1 Write(Y) — T1 TRIES write-lock(Y) => T1 must WAIT for T2
```

Resulting Wait-For Graph:

```
After Step 4:  T2 --> T1          (T2 waits for T1)
After Step 8:  T3 --> T1          (T3 waits for T1)
After Step 9:  T1 --> T2          (T1 waits for T2)

Final Wait-For Graph:
  T1 --> T2 --> (waiting...)
  ^      |
  |      v
  T1 <-- T2 => CYCLE: T1 waits for T2, T2 waits for T1 => DEADLOCK!
  T3 --> T1
```

Resulting Precedence Graph:

```
T1 writes X before T2 reads X:  T1 --> T2
T2 reads Y before T1 reads Y:   (shared, no conflict)
T1 reads Y before T1 writes Y:  (same transaction, skip)
T3 reads/writes Z:              (no conflict with T1, T2 on Z)
T1 writes X before T3 reads X:  T1 --> T3
T2 reads Y before T1 writes Y:  T2 --> T1

Final Precedence Graph:
  T1 --> T2 --> T1  (cycle!) => Not conflict-serializable
  T1 --> T3
```

---

9. Deadlock Recovery

Once a deadlock is detected, the system must take action to break it. There are three key concerns in recovery:

9.1 Victim Selection

The system must choose which transaction(s) to roll back. The goal is to minimize cost. Factors that determine the cost of rolling back a transaction include:

- How long the transaction has been computing and how much longer it would take to complete.
- How many data items the transaction has already used.

- How many more data items the transaction still needs.
- How many other transactions would be involved in the rollback (cascading effect).

9.2 Rollback Strategy

Once a victim is chosen, the system must decide how far to roll it back:

- Total Rollback: Abort the entire transaction and restart it from scratch. This is the simplest approach.
- Partial Rollback: Roll back only as far as necessary to break the deadlock (e.g., release only the locks causing the circular wait). This is more efficient but requires the system to maintain detailed state information about all running transactions.

9.3 Avoiding Starvation

If victim selection is always based purely on cost, the same transaction might be repeatedly chosen as the victim. This transaction would never complete its work — a condition called starvation.

Solution: Include the number of times a transaction has already been rolled back as a factor in the cost calculation. This ensures that a transaction cannot be repeatedly selected as a victim. The most common approach is to count rollbacks and weight them heavily enough that a repeatedly victimized transaction eventually gets priority to complete.

---

10. Summary

This chapter covered the complete landscape of concurrency control in database systems. Below is a structured recap:

Topic	Key Takeaway
Concurrency Problems	Lost Update, Uncommitted Dependency, Inconsistent Analysis arise from concurrent read/writes on the same data
Lock Modes	Shared (S) = read-only, multiple holders allowed; Exclusive (X) = read+write, sole holder
2PL Protocol	Growing phase: acquire locks only. Shrinking phase: release locks only. Guarantees conflict-serializability.
2PL Variants	Strict 2PL holds X-locks till commit (prevents cascading). Rigorous 2PL holds all locks till commit.
Isolation Levels	Serializable (strongest) > Repeatable Read > Read Committed > Read Uncommitted (weakest).

Deadlock Prevention	Wait-Die: older waits, younger dies. Wound-Wait: older wounds younger, younger waits.
Deadlock Detection	Maintain wait-for graph; cycle = deadlock. Periodically check for cycles.
Deadlock Recovery	Select a victim (minimum cost), roll back partially or fully, prevent starvation by tracking rollback count.

— End of Chapter 15: Concurrency Control —